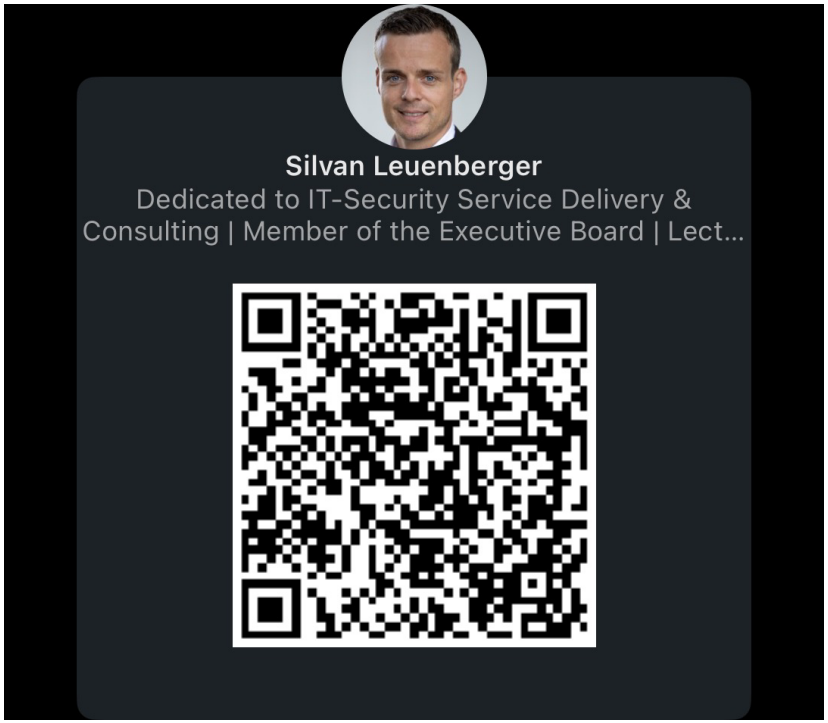
Ausfüllen der Modulevaluation

- Connect with me on linkedIn if you like to



Repetition

- Requirements → NF / F
- Diagraming → Wie zeichnet man dies allgemein und verbindlich / Verständlich auf
- SDLC
- Threat Modelling → Aufzeichnen, tooling → Analyse
- STRIDE → Threat Analyse
- DREAD → Risikobewertung
- UCS / MUCS

→ Vollumfänglicher und Risikobasierter Ansatz ist sehr zielführend und wünschenswert (weil ein solches Vorgehen schont die Ressourcen)

Eliciting Requirements

- Stakeholder analyse
  - Get to know all the involved parties
- Requirement sources
  - Stakeholder
  - Documents
  - Systems
- Requirement type
  - Functional
    - → Login
  - Quality / non-functional
    - → Secure storage of log data
  - General regulations or condition
    - System should be live in Summer
    - PCI-DSS compliant

Types of requirements

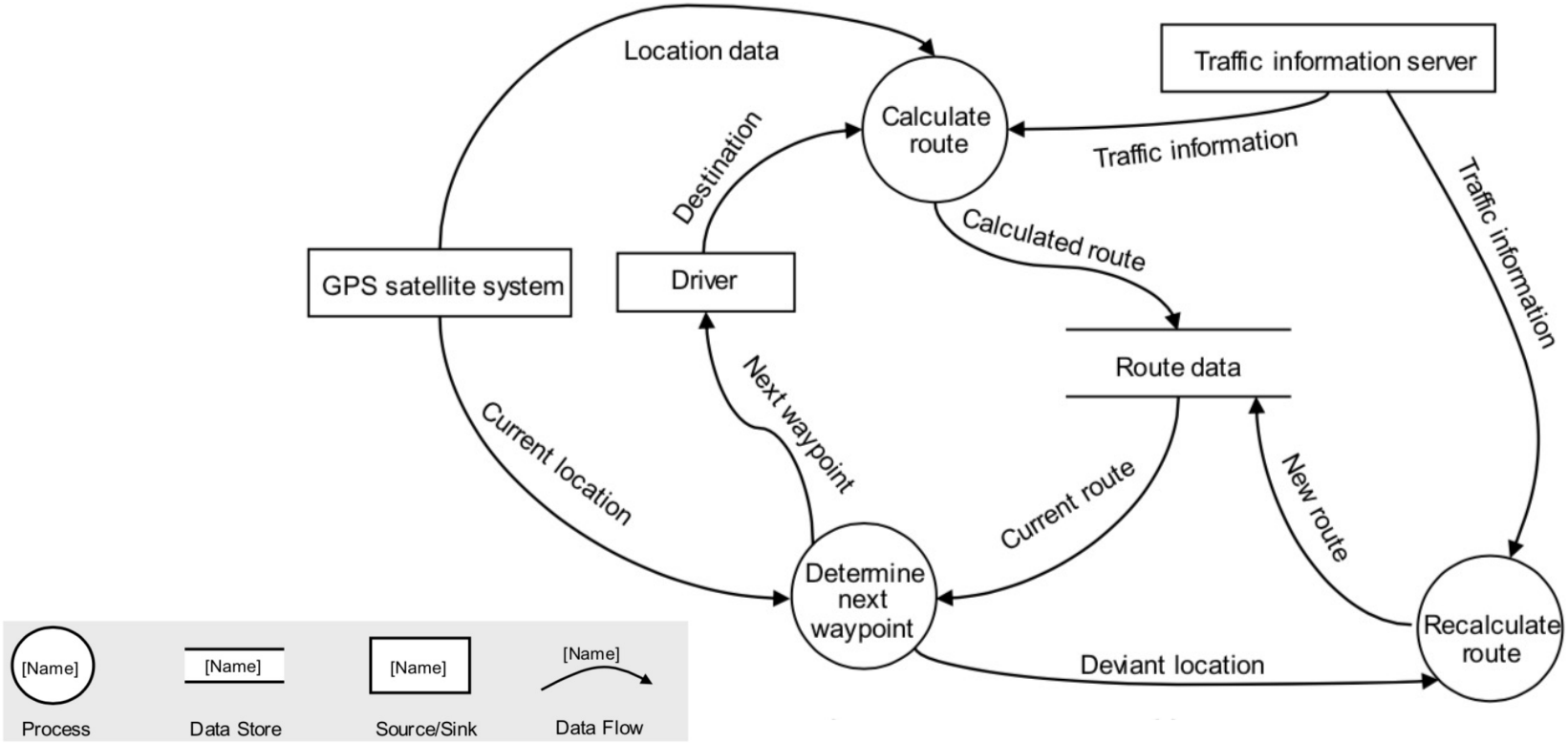
Functional Requirement

- A functional requirement is a requirement concerning a result of behaviour that shall be provided by a function of the system.
- Usually, these requirements are divided into functional requirements, behavioural requirements, and data requirements

Quality Requirement → non functional requirements

- A quality requirement is a requirement that pertains to a quality concern that is not covered by functional requirements.
- Typically, quality requirements are about the performance, availability, dependability, scalability, or portability of a system. Requirements of this type are frequently classified as non-functional requirements.

Remember: Dataflow diagram (DFD)

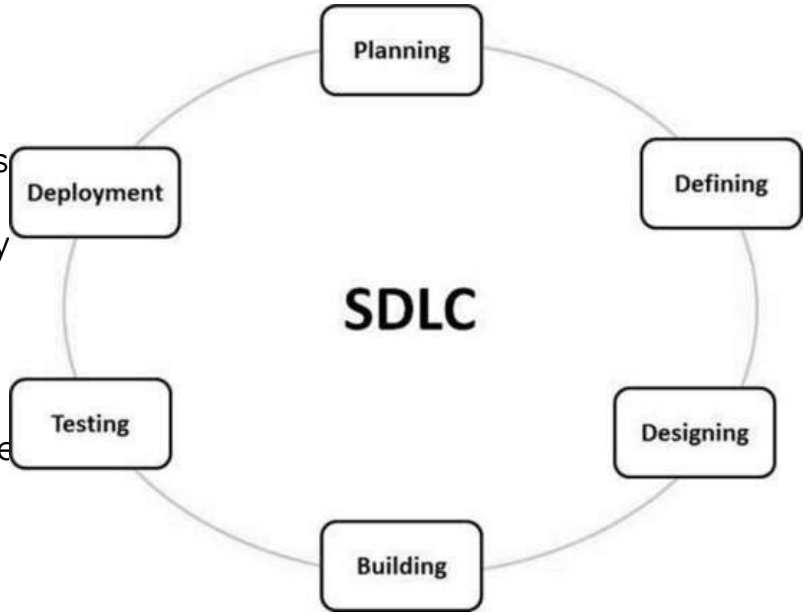


[Requirements Engineering Fundamentals], p80 ff

SDLC

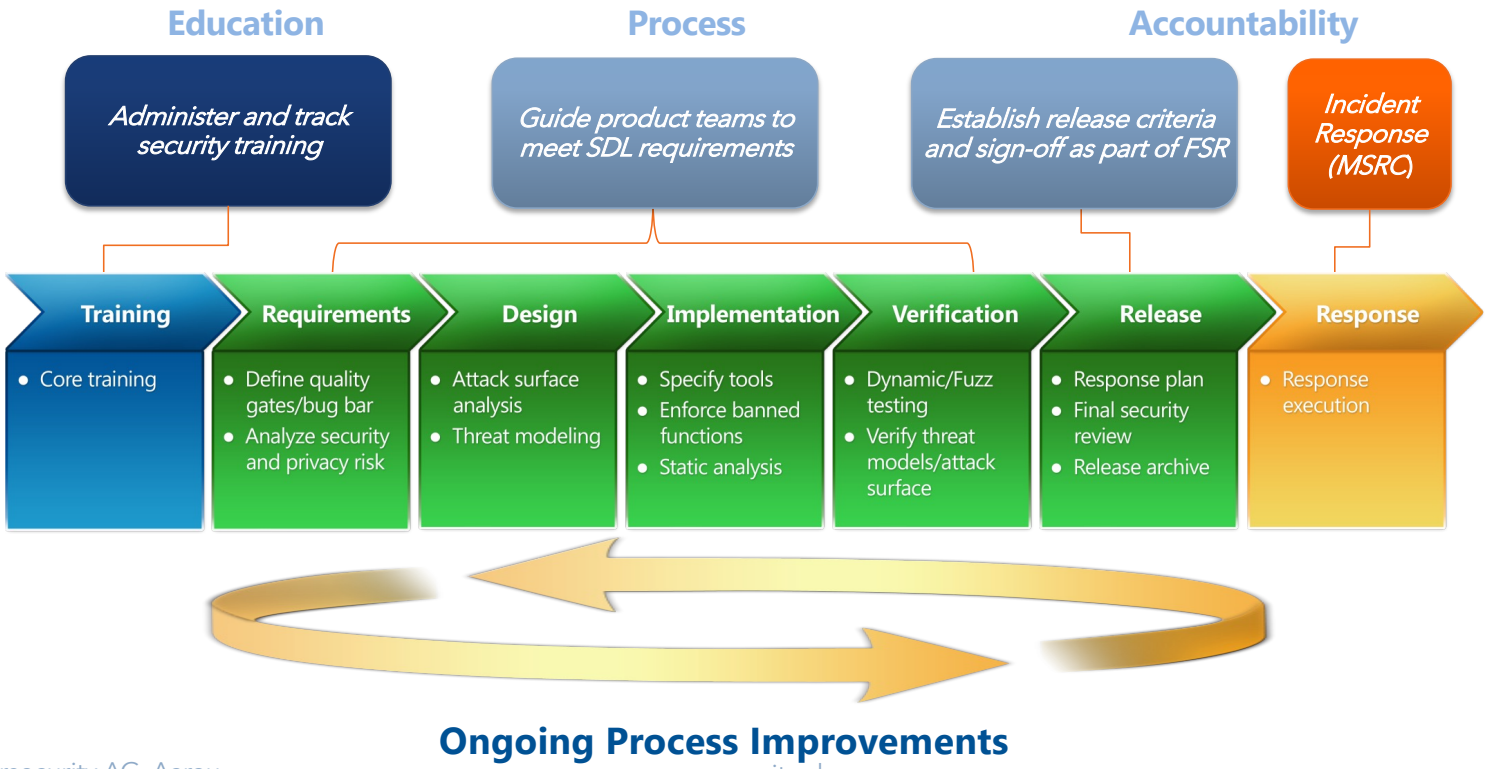
- The software development life cycle (SDLC) is a framework defining tasks performed at each step in the software development process. SDLC is a structure followed by a development team within the software organization. It consists of a detailed plan describing how to develop, maintain and replace specific software. The life cycle defines a methodology for improving the quality of software and the overall development process.

The software development life cycle is also known as the software development process.



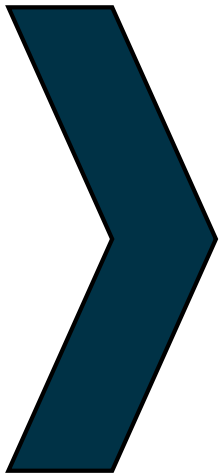
Working to protect our users…

The following slides concerning MSSDLC have been take from:
https://www.microsoft.com/en-us/securityengineering/sdl/



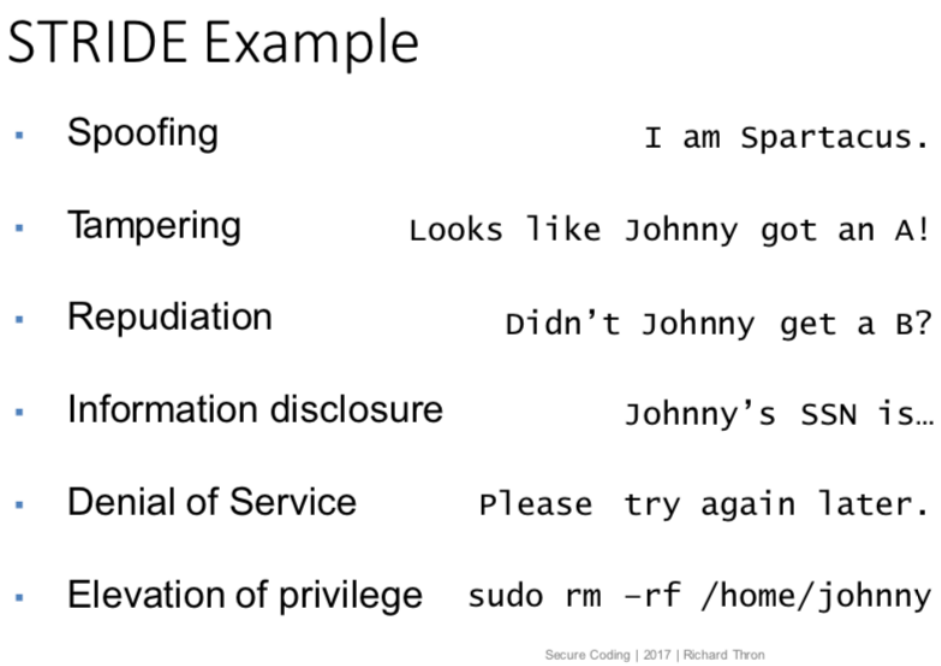
STRIDE → Identify/Enumerate Threats

Threat		Property we want
• Spoofing		• Authentication
• Tampering		• Integrity
• Repudiation		• Nonrepudiation
• Information Disclosure		• Confidentiality
• Denial of Service		• Availability
• Elevation of Privilege		• Authorization



Use STRIDE to step through the diagram elements

STRIDE Example



DREAD – How to rate the RISK

A methodology for **risk rating**

Each threat is graded in this categories:

- **D**amage - how bad would an attack be?
- **R**eproducibility - how easy is it to reproduce the attack?
- **E**xploitability - how much work is it to launch the attack?
- **A**ffected users - how many people will be impacted?
- **D**iscoverability - how easy is it to discover the threat?

Some examples

Rating	High (3)	Medium (2)	Low (1)
Damage potential	The attacker can undermine the security system; get full trust authorization; run as administrator; upload content.	Leaking sensitive information	Leaking trivial information
Reproducibility	The attack can be reproduced every time and does not require a timing window.	The attack can be reproduced, but only with a timing window and a particular race situation.	The attack is very difficult to reproduce, even with knowledge of the security hole.

DREAD - Calculation

Risk rating	Result
High	12 – 15
Medium	8 -11
Low	5 – 7

Threat	D	R	E	A	D	Total	Rating
Attacker obtains authentication credentials by monitoring the network.	3	3	2	2	2	12	High
SQL commands injected into application.	3	3	3	3	2	14	High

Some security experts feel that including the "Discoverability" element as the last D rewards **Security through obscurity**, so some organizations have either moved to a **DREAD-D** "DREAD minus D" scale (which ignores Discoverability) or always assume that Discoverability is at its maximum rating.

Analysis with use cases

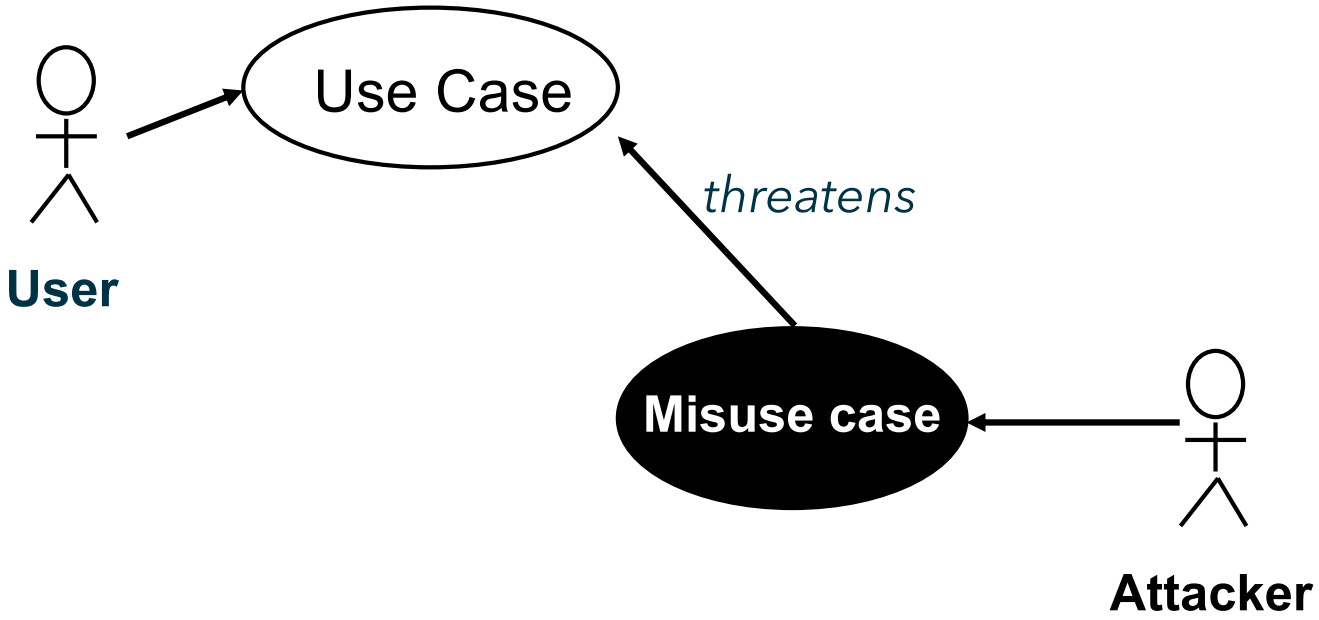
- Requirements analysis with use case diagrams can be done by carrying out the following steps:
  - Draw a context model
  - Write a glossary
  - Detail the context model

For Security: Add Misuse Cases

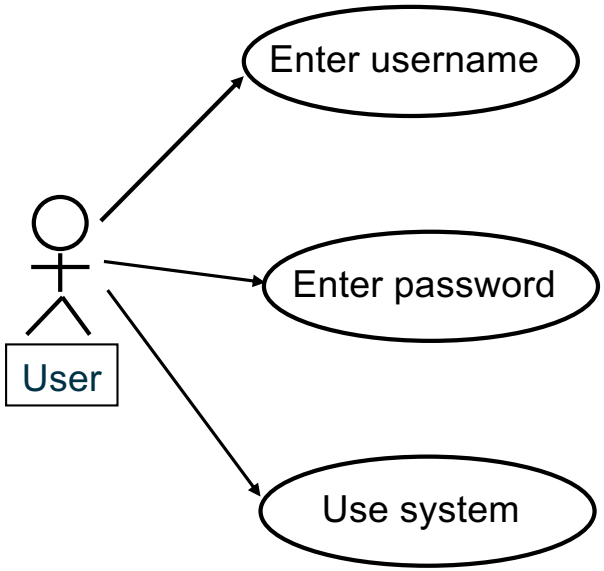
UC vs. MUC

- UC
  - Wanted
  - Should happen in the System
  - Good for functional requirement
- MUC
  - **UN**-Wanted
  - Should **NOT** happen in the System
  - Focus on **security** requirement

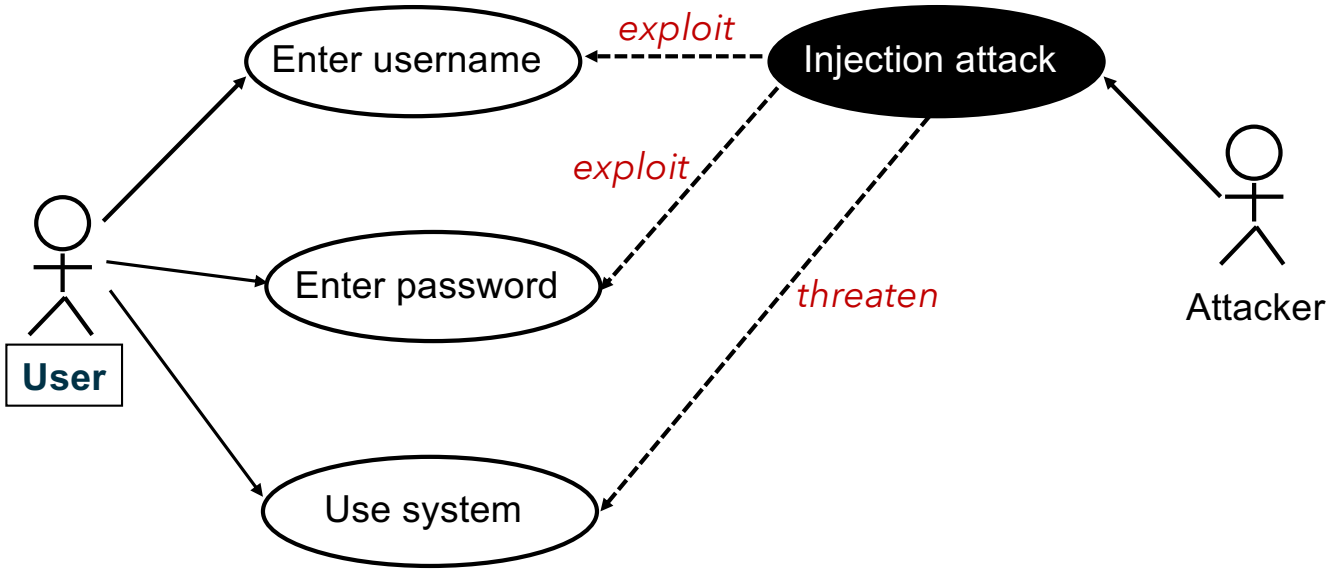
Security considerations in analysis



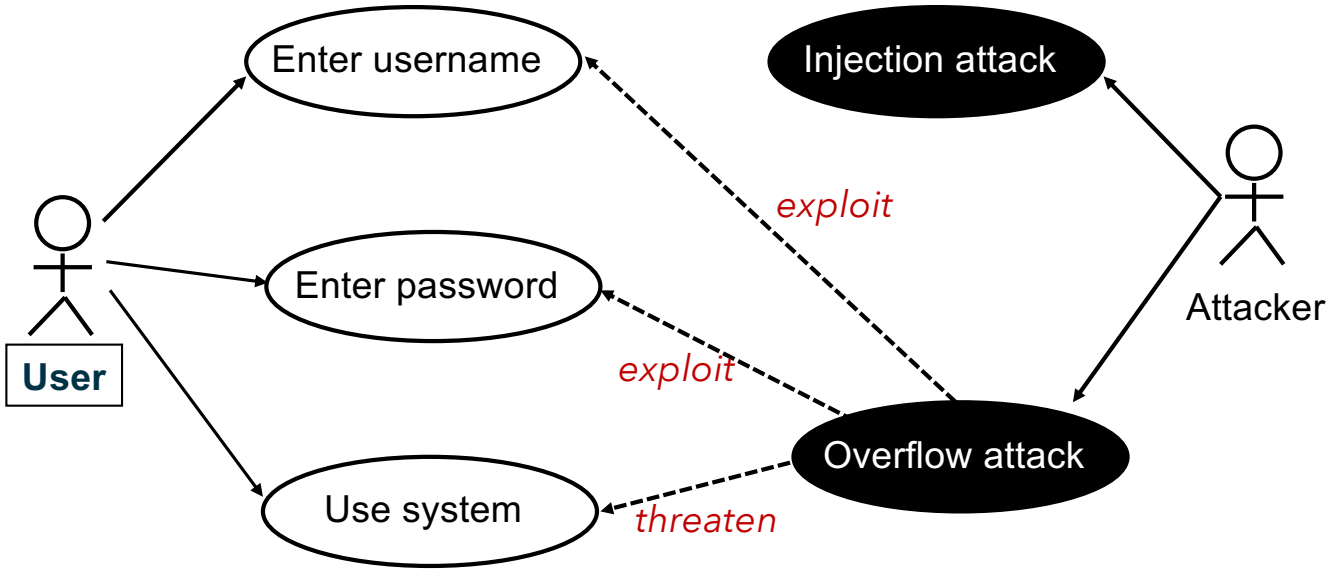
Example: Misuse Cases



Example: Misuse Cases

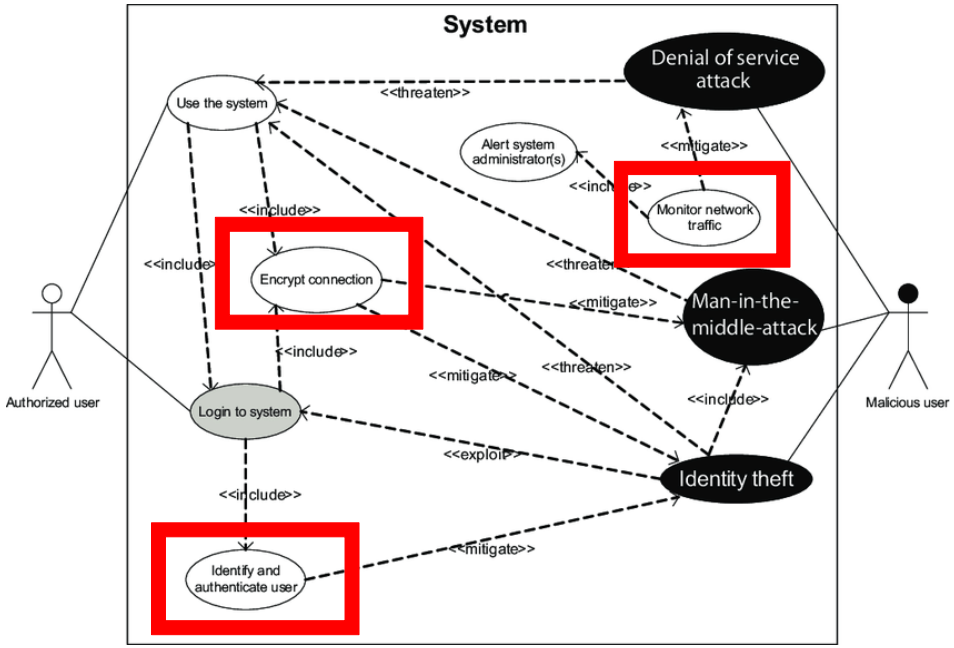


Example: Misuse Cases



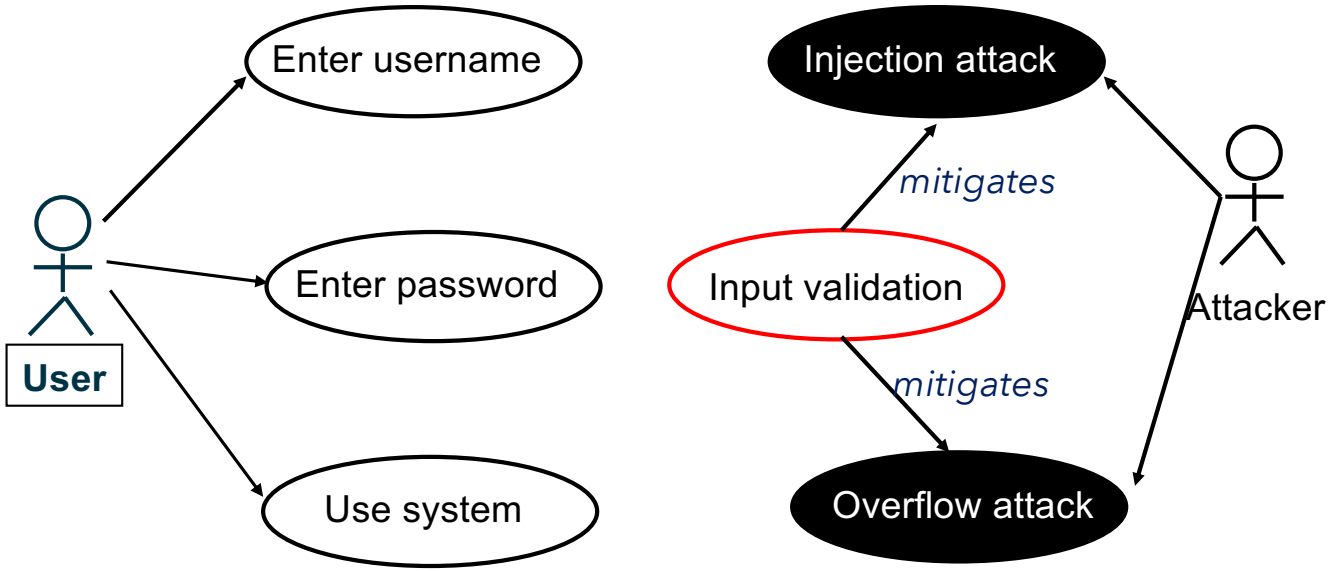
Mitigation

- Insert mitigation use cases
  - that will mitigate the thread of the misuse case
- Describe the mitigation use cases either
  - as part of the misuse case description or
  - as a use case of its own (better)

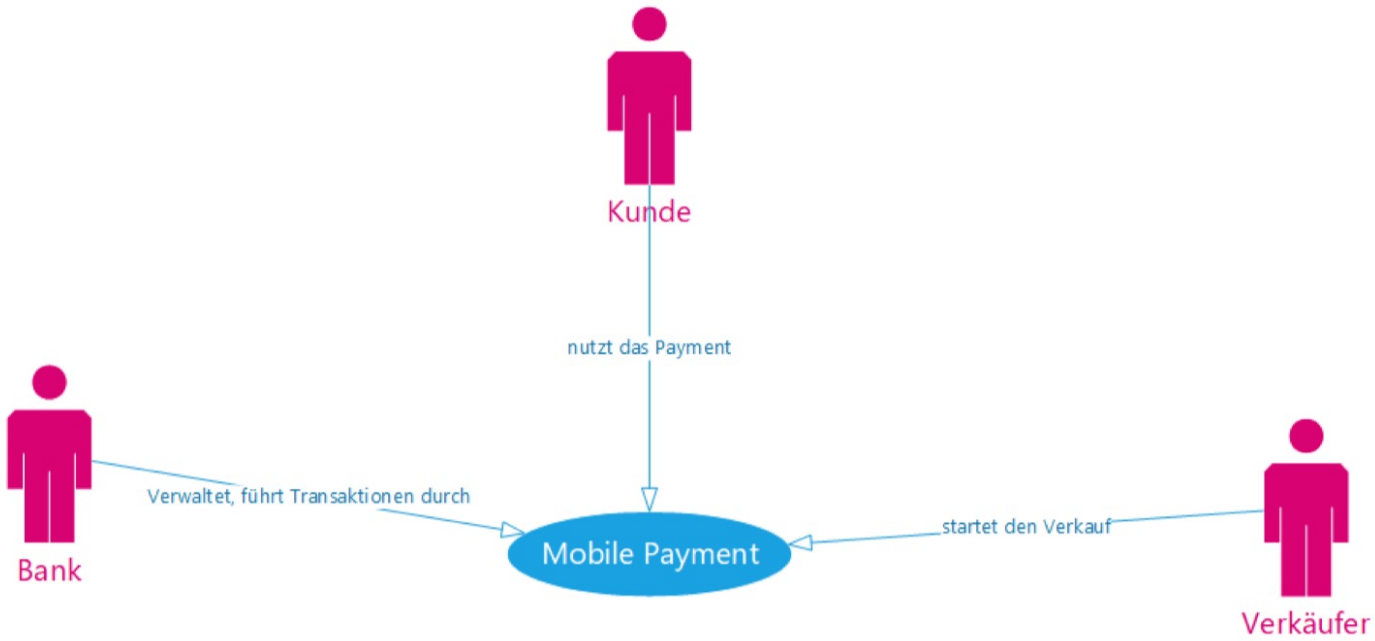


https://www.researchgate.net/publication/268418932_Security_Testing_of_Web_Based_Applications

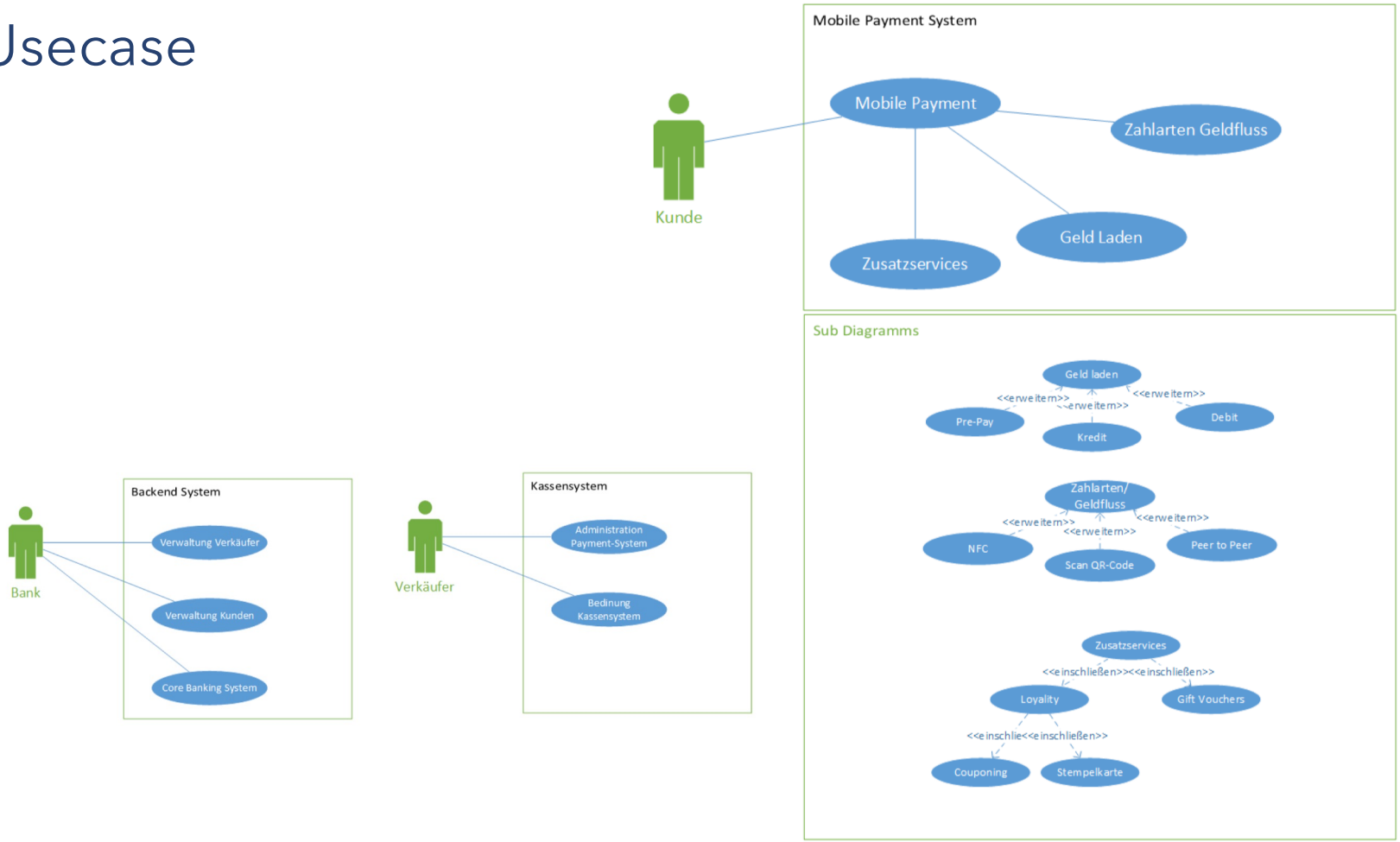
Example Mitigation Use Cases



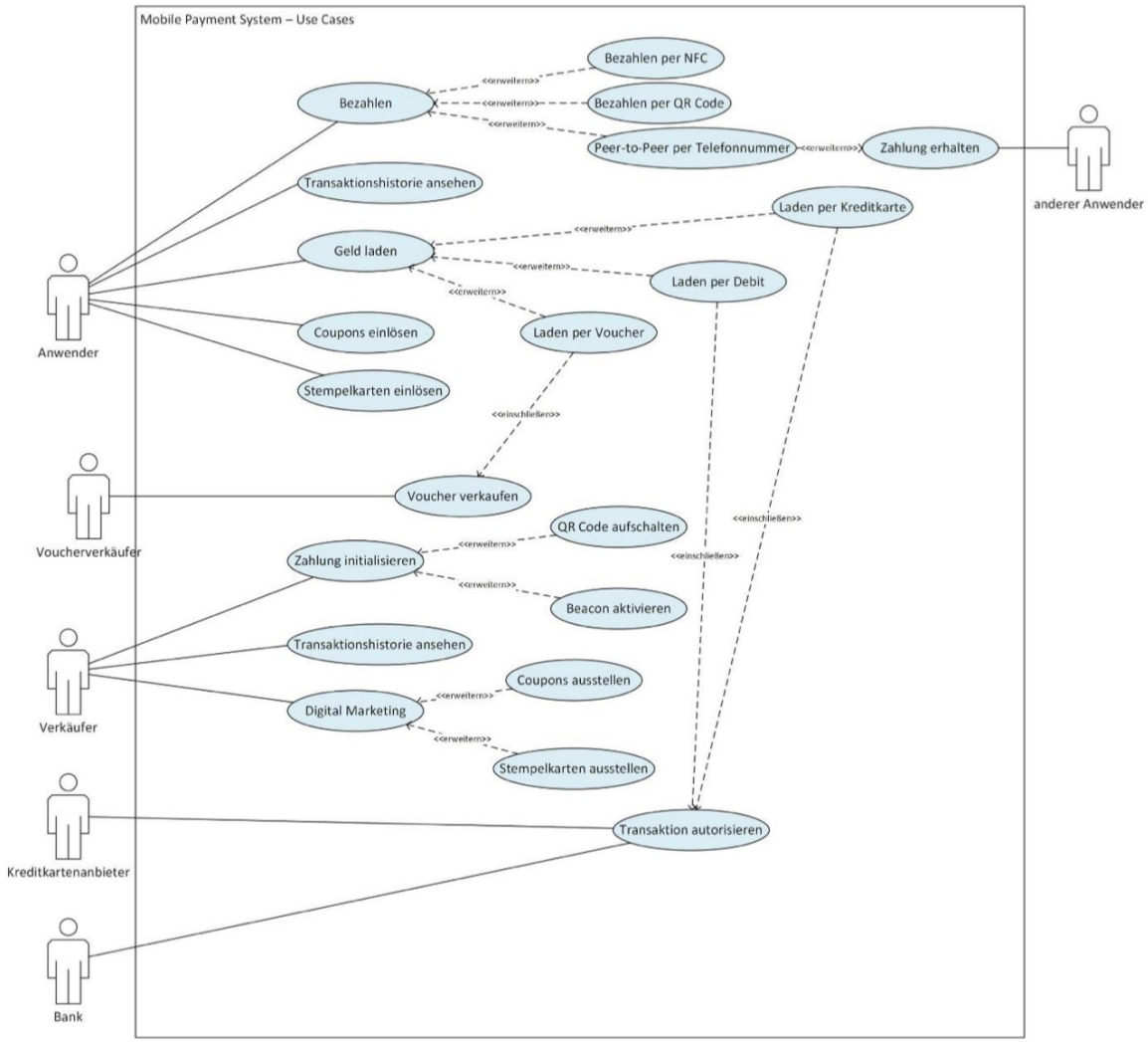
Context Diagram



Usecase


Mobile Payment System
Mobile Payment
Zahlarten Geldfluss
Geld Laden
Zusatzservices
Kunde
Sub Diagramms
Geld laden
<<erweitern>>
<<erweitern>>
<<erweitern>>
Pre-Pay
Kredit
Debit
Zahlarten/ Geldfluss
<<erweitern>>
<<erweitern>>
<<erweitern>>
NFC
Scan QR-Code
Peer to Peer
Zusatzservices
<<einschließen>><<einschließen>>
Loyality
Gift Vouchers
<<einschlie<<einschließen>>
Couponing
Stempelkarte
Backend System
Verwaltung Verkäufer
Verwaltung Kunden
Core Banking System
Bank
Kassensystem
Administration Payment-System
Bedinung Kassensystem
Verkäufer

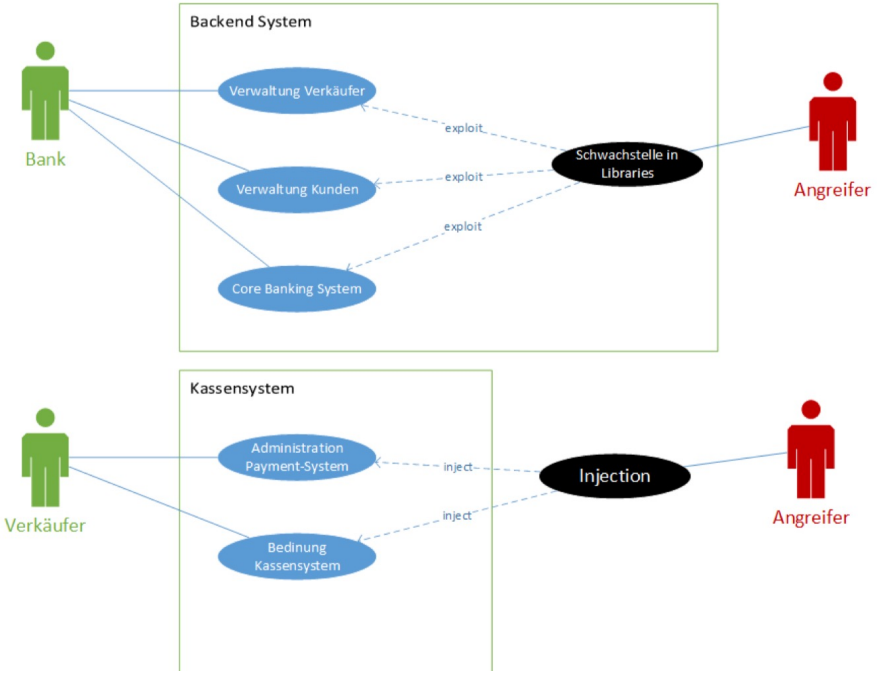
Usecase Diagra

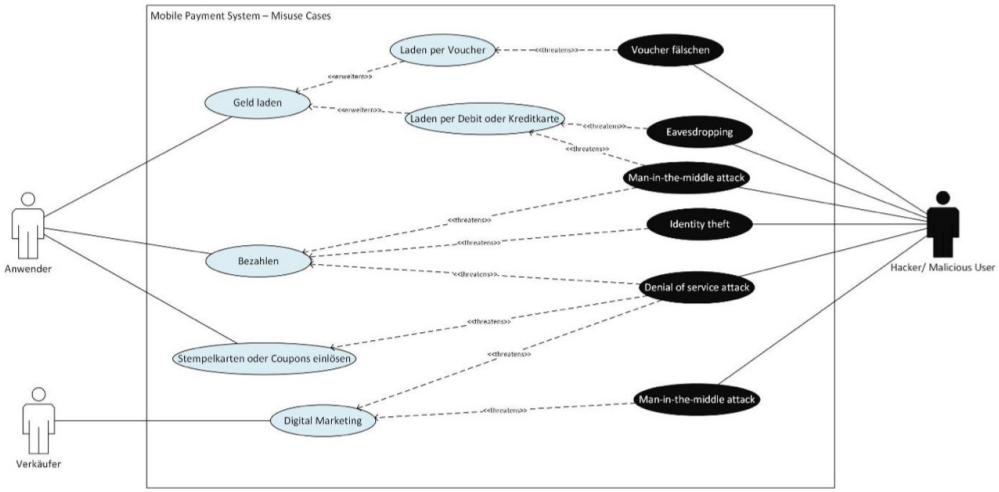


Usecase-Tabelle

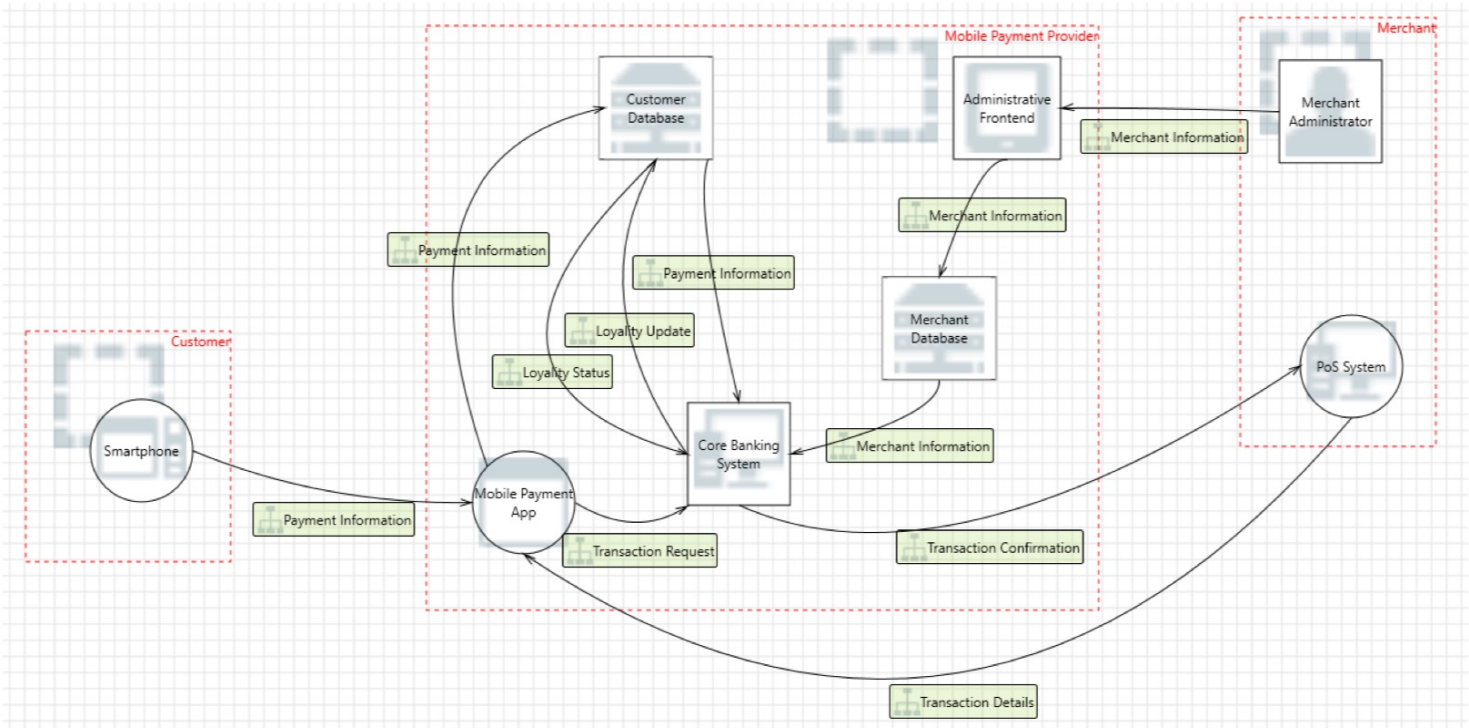
Peer-to-Peer Bezahlung	
Auslösender Aktor	Anwender
Zweck	Beschreibt eine Peer-to-Peer Zahlung zwischen zwei Anwendern, die jeweils eine App besitzen.
Precondition	Der Anwender will eine Zahlung an einen anderen Anwender auslösen und die Telefonnummer des anderen Nutzers ist bekannt. Die App ist im Ruhemodus oder ausgeschaltet.
Postcondition	Die Zahlung wurde ausgeführt und bestätigt. Evtl. App geht in definierten Zustand, muss für nächste Transaktion erst wieder scharf gemacht werden.
Ablauf	1. Der Anwender startet die Mobile Payment Applikation. 2. Der Anwender wählt die Zahlungsart Peer-to-Peer. 3. Der Anwender wählt die Telefonnummer des Empfängers und den Zahlungsbetrag. 4. Die App sendet den Zahlungsinformation an das Backend. 5. Das Backend prüft ob genügend Geld auf der App geladen ist und Empfänger als Kunde registriert ist. 6. Das Backend schickt die Zahlungsdaten an die App zur Bestätigung durch den Anwender. 7. Anwender bestätigt die Zahlung, App sendet OK an Backend. 8. Das Backend schreibt die Zahlung in der App des Empfängers gut. 9. Das Backend sendet die Zahlungsbestätigung an die App des Anwenders und die Information über die erfolgte Zahlung auf die App des Empfängers. 10. Der Anwender sieht die Zahlungsbestätigung auf seiner App. 11. Der Empfänger sieht die Zahlungsbestätigung auf seiner App.
Erweiterungen	Für Schritt 5: Falls nicht genügend Guthaben auf der App vorhanden ist oder der Anwender in Punkt 7 nicht innerhalb einer bestimmten Zeit die Zahlung bestätigt hat, wird die Zahlung abgelehnt und die entsprechende Information weitergeleitet.

Misusecase

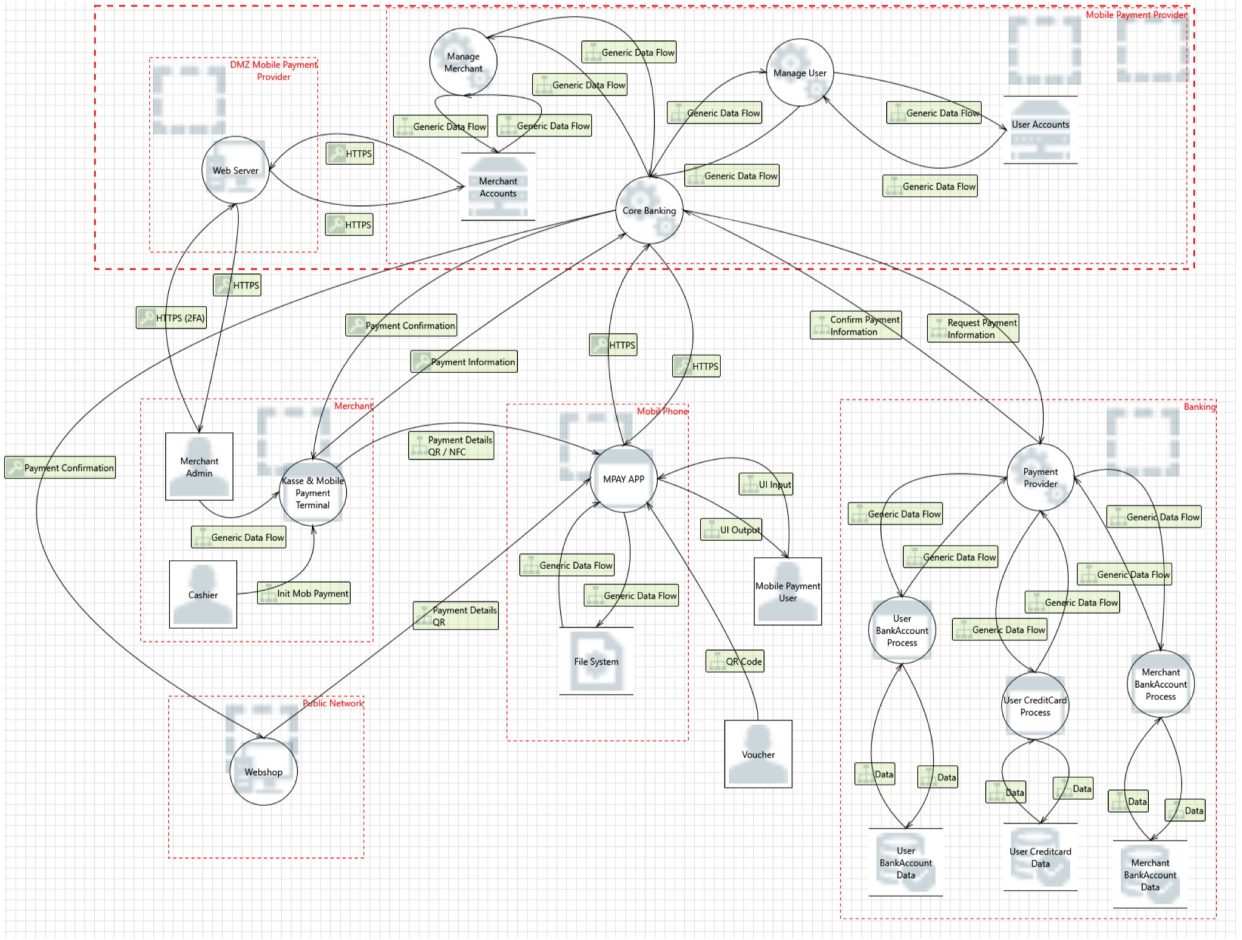

Backend System
Verwaltung Verkäufer
Verwaltung Kunden
Core Banking System
Schwachstelle in Libraries
exploit
exploit
exploit
Bank
Angreifer
Kassensystem
Administration Payment-System
Bedinung Kassensystem
Injection
inject
inject
Verkäufer
Angreifer


Mobile Payment System – Misuse Cases
Laden per Voucher
Voucher fälschen
Geld laden
Laden per Debit oder Kreditkarte
Eavesdropping
Man-in-the-middle attack
Identity theft
Bezahlen
Denial of service attack
Stempelkarten oder Coupons einlösen
Digital Marketing
Man-in-the-middle attack
Anwender
Verkäufer
Hacker/ Malicious User

Threatmodeling



Threatmodeling



Threatmodeling